

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

LAB 2 - WEB APPLICATION SECURITY TESTING

BY

MUSTAPHA ABUBAKAR

ID: C11/26/FCDF/17172

Course WADF-2026-M04 – LAB 2 - WEB APPLICATION SECURITY TESTING

INSTRUCTOR

MR AMINU IDRIS

Date of Submission 06 SEPTEMBER, 2026

INTRODUCTION

This report documents the completion of Lab 2 Web Application Security Testing (DVWA and Mutillidae), carried out on an authorised, isolated lab environment consisting of a Kali Linux virtual machine (192.168.80.130) and an instructor-authorised Metasploitable 2 target (192.168.80.128). Using Burp Suite, Nikto, DIRB, and manual SQL injection testing, the practical examined DVWA's file upload feature and Mutillidae's SQL-based login form, uncovering weak upload validation, a publicly accessible upload directory, an exposed phpMyAdmin panel, and unsanitised SQL input that broke query syntax under manual testing. These findings build directly on the network and service reconnaissance from Lab 1, shifting focus from what services are exposed to how the application itself handles user-controlled input.

AIM

The aim of this lab is to assess how the DVWA and Mutillidae training applications handle HTTP requests, file uploads, content discovery, and user-supplied input, using Burp Suite, Nikto, DIRB and a controlled manual/automated SQL injection exercise, and to produce defensive recommendations based on the findings.

OBJECTIVES

- Explain HTTP requests/responses, methods, headers, cookies and parameters.
- Use browser Developer Tools and Burp Suite to inspect traffic.
- Assess a DVWA file-upload feature using harmless files.
- Understand extension, MIME, storage and execution controls.
- Use Nikto for web-server configuration observations.
- Use DIRB for content/directory discovery.
- Identify user-controlled input in Mutillidae.

- Perform basic manual SQL injection observations in the training app.
- Use SQLMap only to verify and enumerate approved lab metadata.
- Write defensive recommendations.

TOOLS AND RESOURCES USED

- ❖ Kali GNU/Linux Virtual Machine
- ❖ DVWA (Damn Vulnerable Web Application) - authorised training target
- ❖ Mutillidae - authorised training target
- ❖ Burp Suite (Community Edition)
- ❖ Nikto
- ❖ DIRB
- ❖ SQLMap
- ❖ Firefox Developer Tools

METHODOLOGY

The methodology below follows the exact sequence defined in the instructor's laboratory brief.

Each step records the command executed, its purpose, the expected result, and the supporting screenshot evidence.

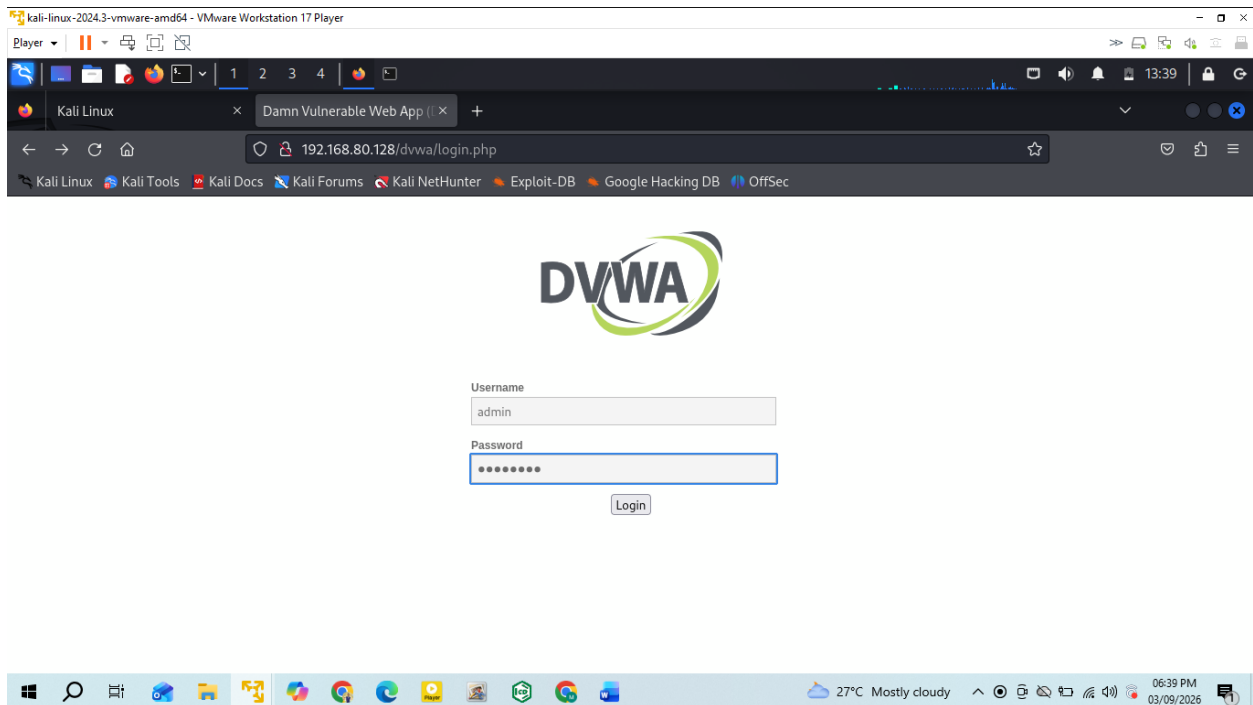
Part 1 - Open DVWA

This part re-uses the target already identified in Lab 1 and confirms the DVWA application is reachable, without repeating Lab 1's network reconnaissance.

Step 1: Confirm the application URL

Command used

`http://<target-ip>/dvwa/`



The command was executed successfully as shown in Figure 1 above. Open the authorised DVWA path in Firefox. The DVWA login/home page loads.

Step 2: Set DVWA security to Low

Command used

DVWA Security tab -> select Low -> Submit



The command was executed successfully as shown in Figure 2 above. Setting security to Low removes extra protections so the exercise behaves consistently for a beginner lab. The Low setting remains selected after submission.

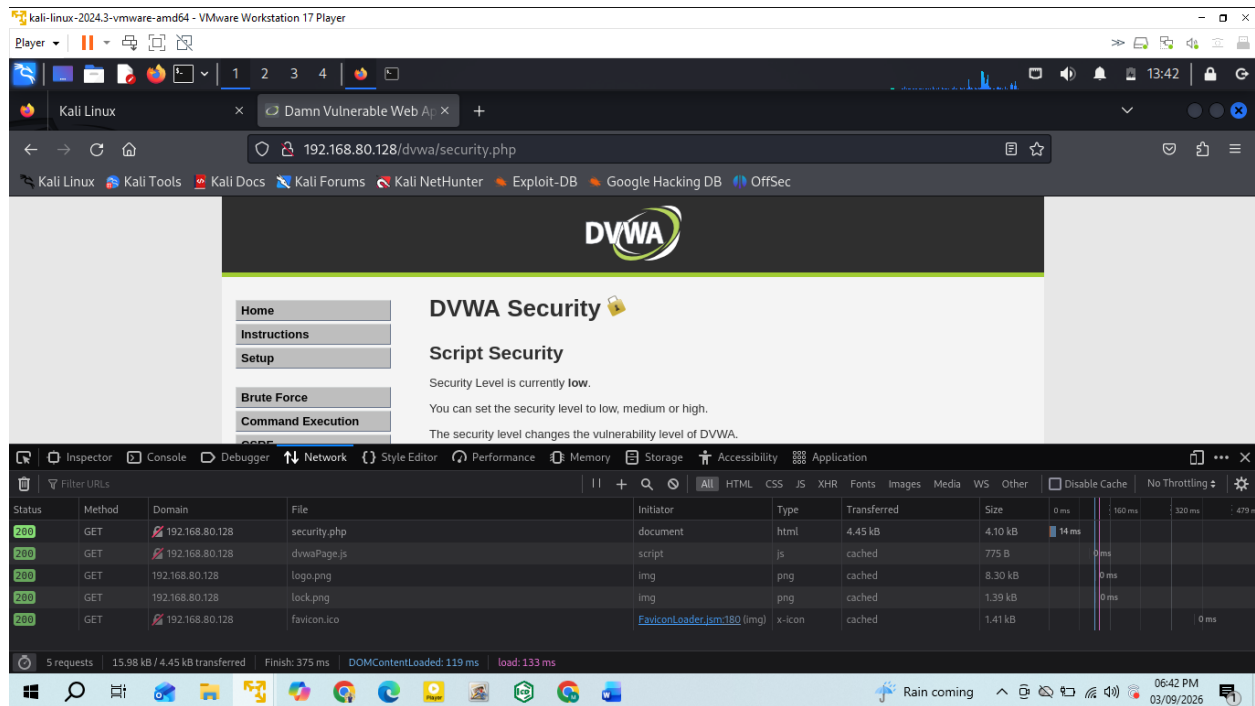
Part 2 - Browser Developer Tools

Before using Burp Suite, a normal browser request was inspected directly using the browser's built-in Network tools.

Step 3: Open Network tools

Command used

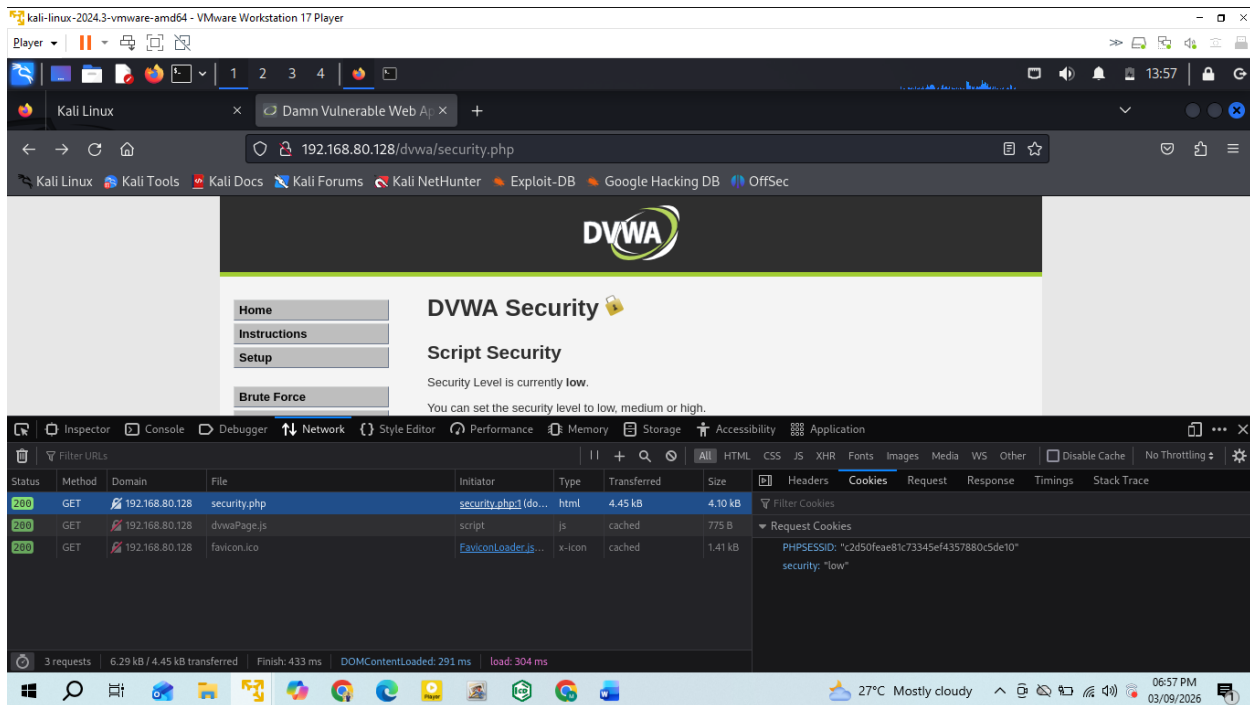
F12 -> Network tab -> reload DVWA -> select one request



The command was executed successfully as shown in Figure 3 above. The Network panel lists every request the browser makes, with method, URL, status, and headers. Method, URL, status and headers for the selected request.

Step 4: Identify cookies

Command used



The command was executed successfully as shown in Figure 4 above. Cookie names indicate what session state the browser is holding. Do not copy secret cookie values into the report. One or more cookie names are visible.

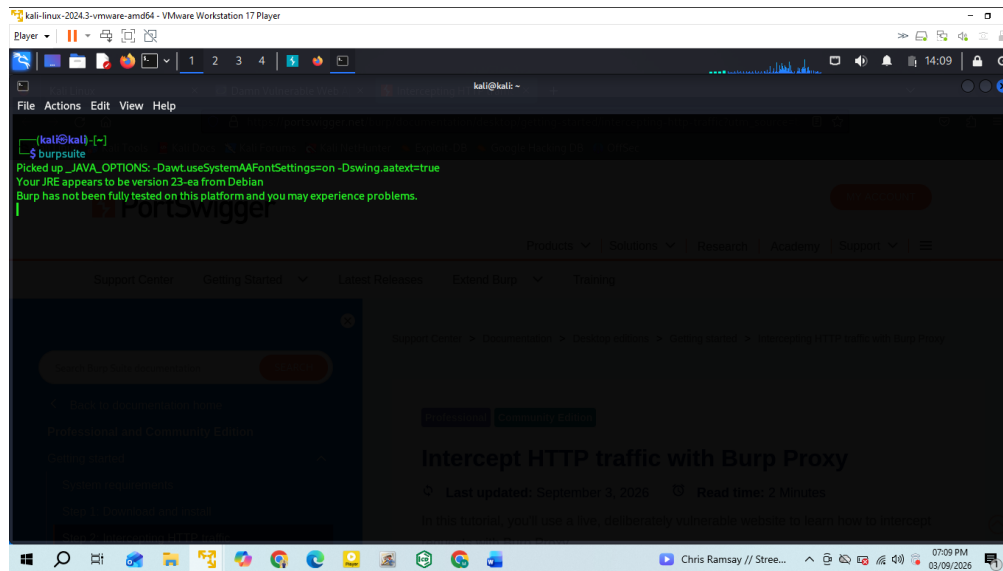
Part 3 - Burp Suite

Burp Suite was used as an intercepting proxy to inspect and understand raw HTTP requests generated by DVWA.

Step 5: Start Burp

Command used

Burpsuite

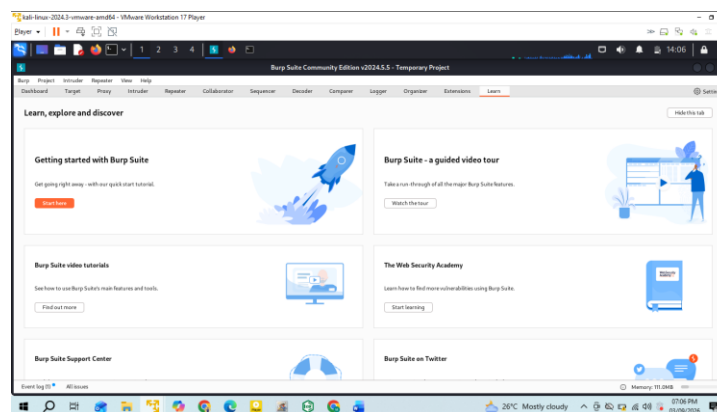


The command was executed successfully as shown in Figure 5 above. A temporary project was used for this lab session. The Proxy tab is available.

Step 6: Open Burp browser

Command used

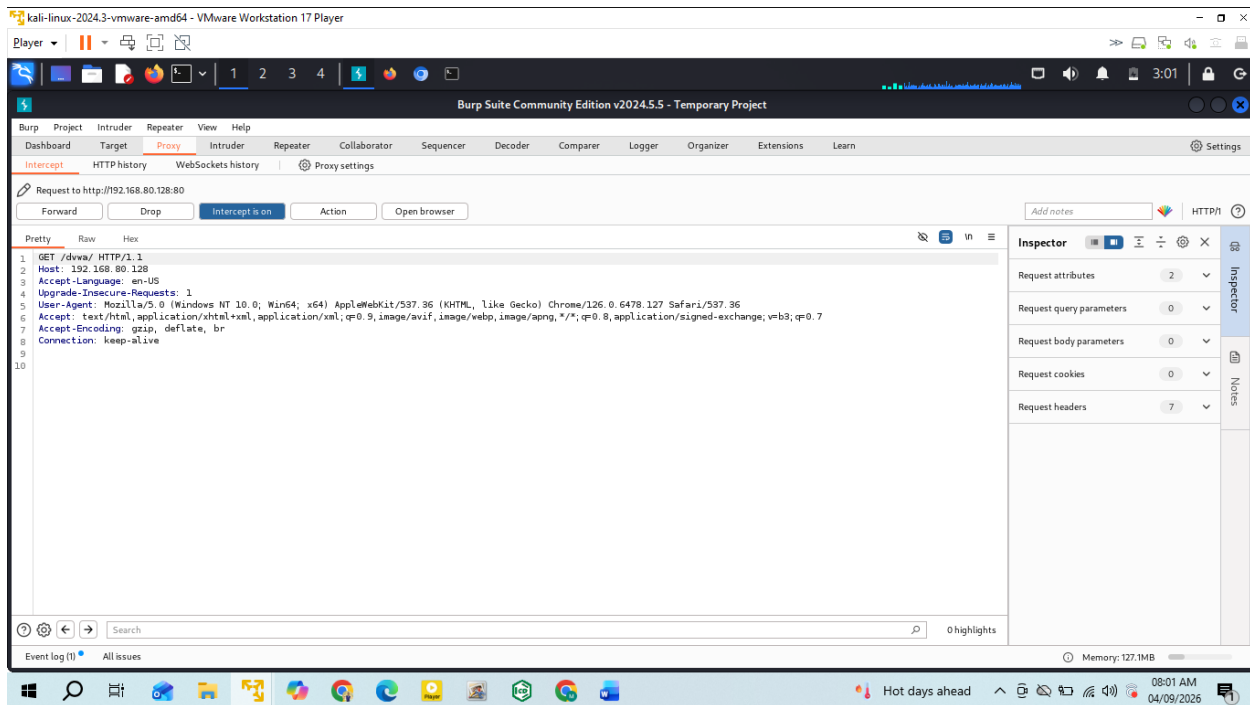
Proxy -> Intercept -> Open Browser



The command was executed successfully as shown in Figure 6 above. This opens a browser already pre-configured to route traffic through Burp. A browser window opens ready for use.

Step 7: Intercept a harmless request

Command used



The command was executed successfully as shown in Figure 7 above. The intercepted request line, Host, User-Agent and Cookie headers were identified before forwarding the request on. The page loads normally once forwarded.

PART 4 - DVWA FILE UPLOAD

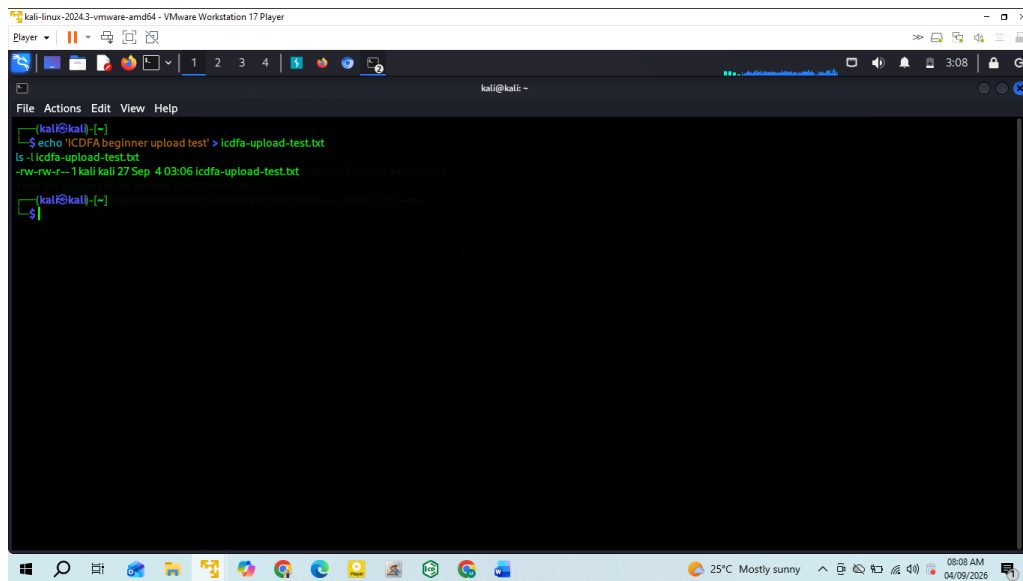
A harmless text file was used throughout to test the File Upload module's behaviour, structure and validation.

Step 8: Create a harmless file

Command used

```
echo 'ICDFA beginner upload test' > icdfa-upload-test.txt
```

```
ls -l icdfa-upload-test.txt
```

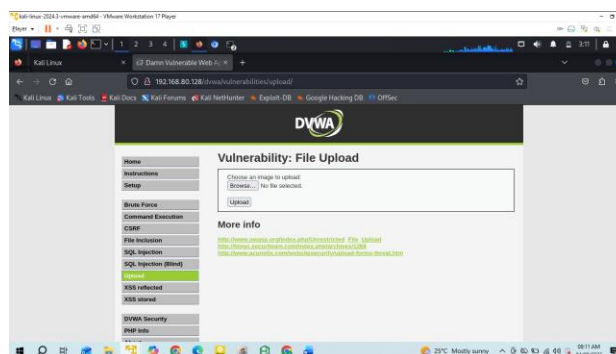


The command was executed successfully as shown in Figure 8 above. This creates a small plain-text file containing no executable content. A small .txt file is created.

Step 9: Open File Upload module

Command used

DVWA -> File Upload

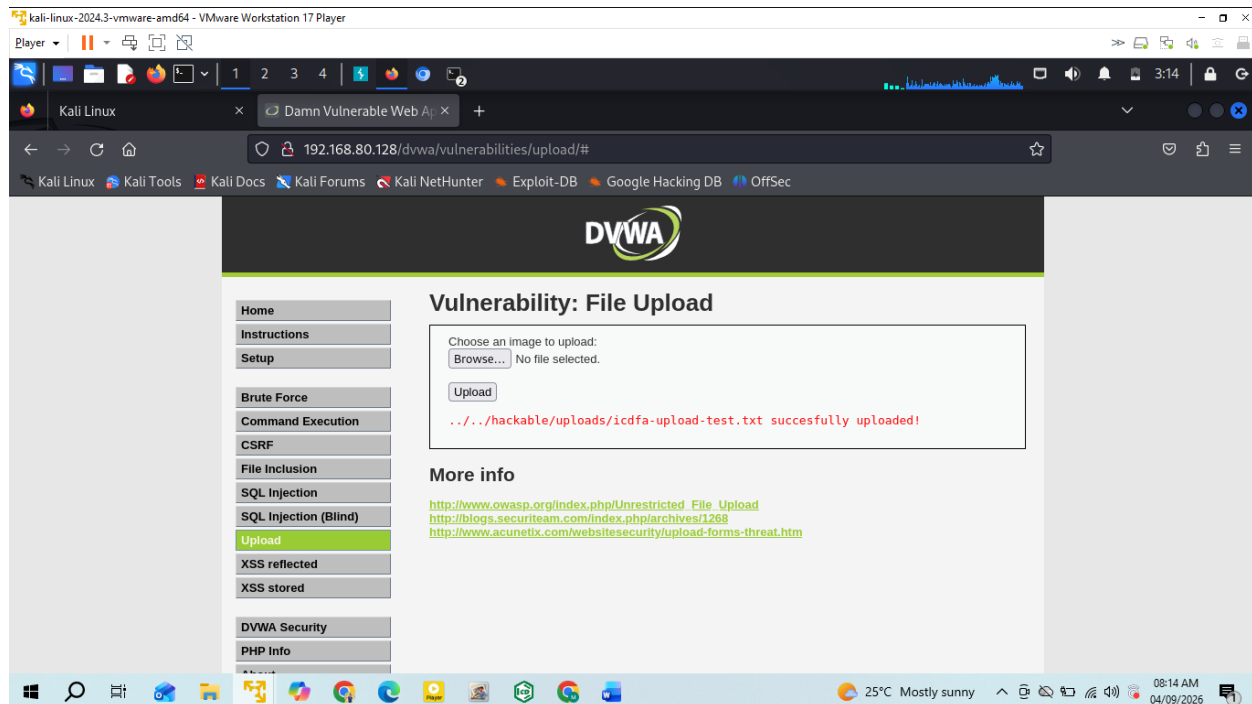


The command was executed successfully as shown in Figure 9 above. This navigates to the module under test. The upload form appears.

Step 10: Normal upload

Command used

Upload icdfa-upload-test.txt (Intercept off)

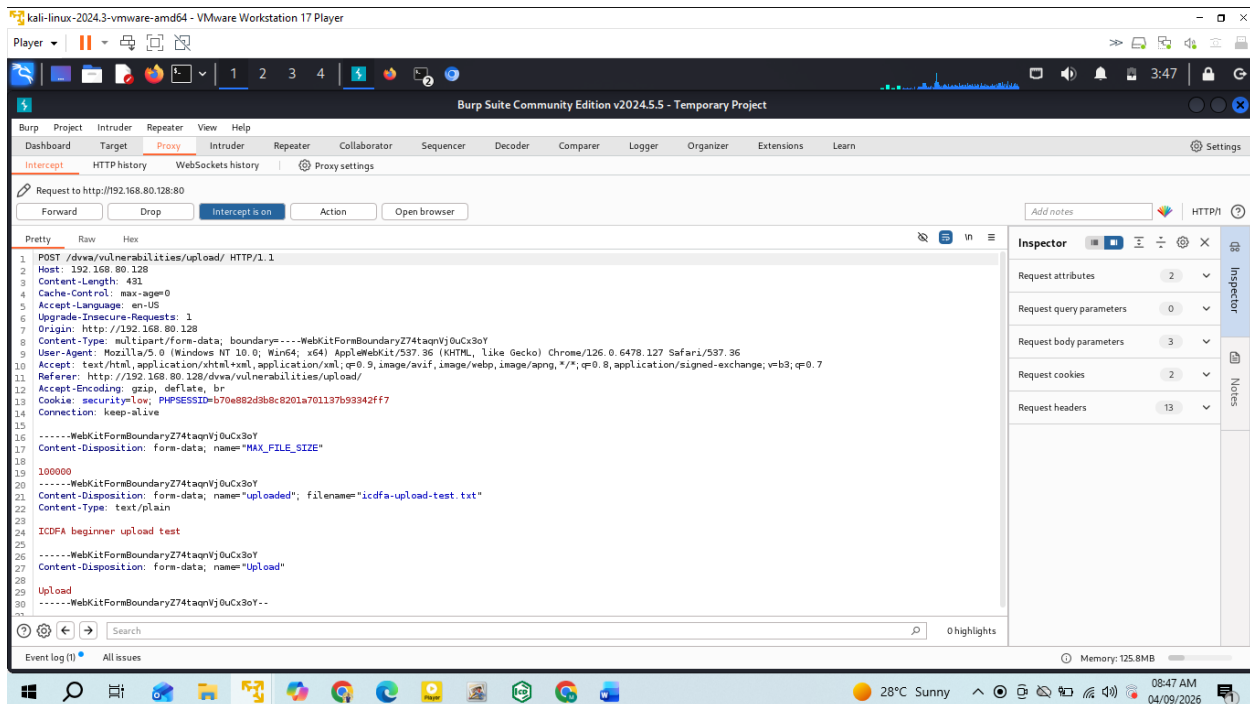


The harmless .txt file was uploaded without any modification, and the response confirmed success: .../hackable/uploads/icdfa-upload-test.txt successfully uploaded! as shown in Figure 10. This is a significant baseline finding — despite the label reading "Choose an image to upload," the application accepted a plain text file with no image content, indicating it is not actually restricting uploads to image types at this security level.

Step 11: Intercept upload

Command used

Intercept ON -> upload again -> inspect POST body



The upload request was intercepted in Burp, as shown in Figure 11. The request body uses multipart/form-data encoding, with each part separated by a boundary string. The file part shows Content-Disposition: form-data; name="uploaded"; filename="icdfa-upload-test.txt" followed by Content-Type: text/plain and the raw file content itself. This confirms exactly how a browser packages a file for upload, and shows the filename and MIME type are both supplied by the client, not verified independently by the browser.

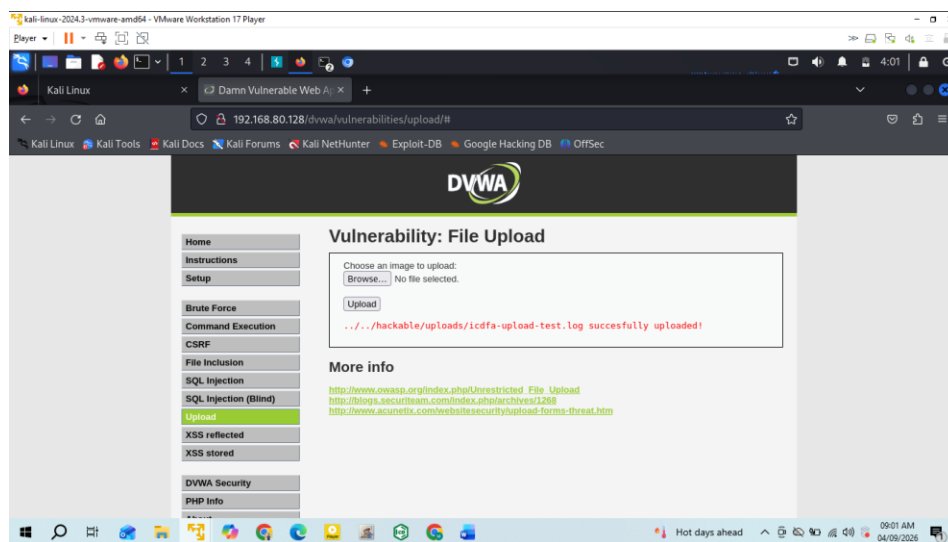
PART 5 - SAFE FILE VALIDATION EXPERIMENTS

Two controlled experiments were carried out to test how the application validates uploaded files, changing only the extension and only the declared MIME type in turn - never the file's actual content.

Step 12: Extension handling

Command used

```
cp icdfa-upload-test.txt icdfa-upload-test.log
```

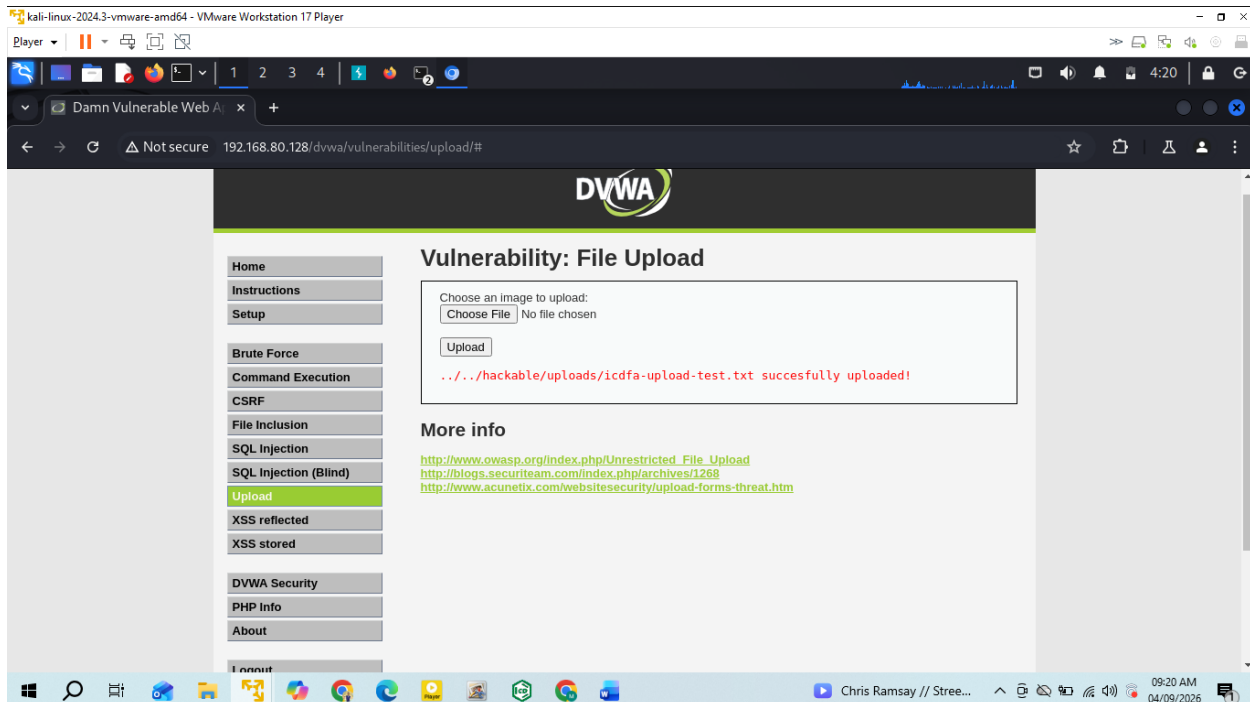
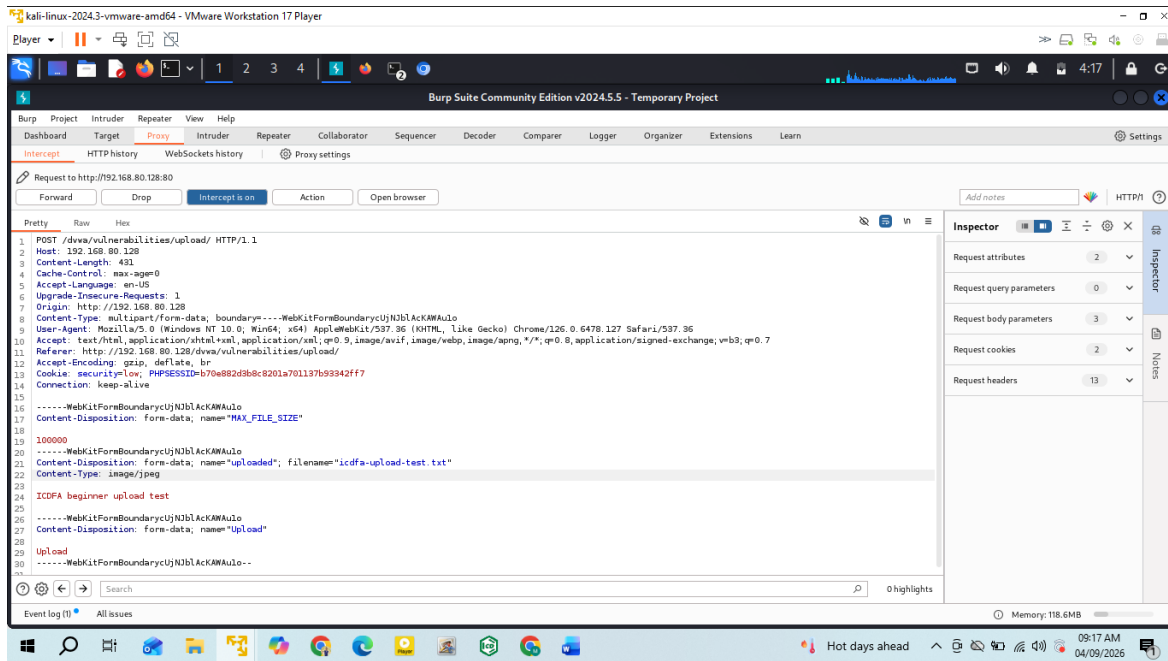


The .log copy uploaded successfully with the same response as the .txt file: `.../hackable/uploads/icdfa-upload-test.log successfully uploaded!` as shown in Figure 12. Comparing this with Step 10, both .txt and .log extensions were accepted identically, showing this DVWA security level (Low) performs no extension-based filtering at all — any file extension appears to pass through unchecked.

Step 13: MIME handling

Command used

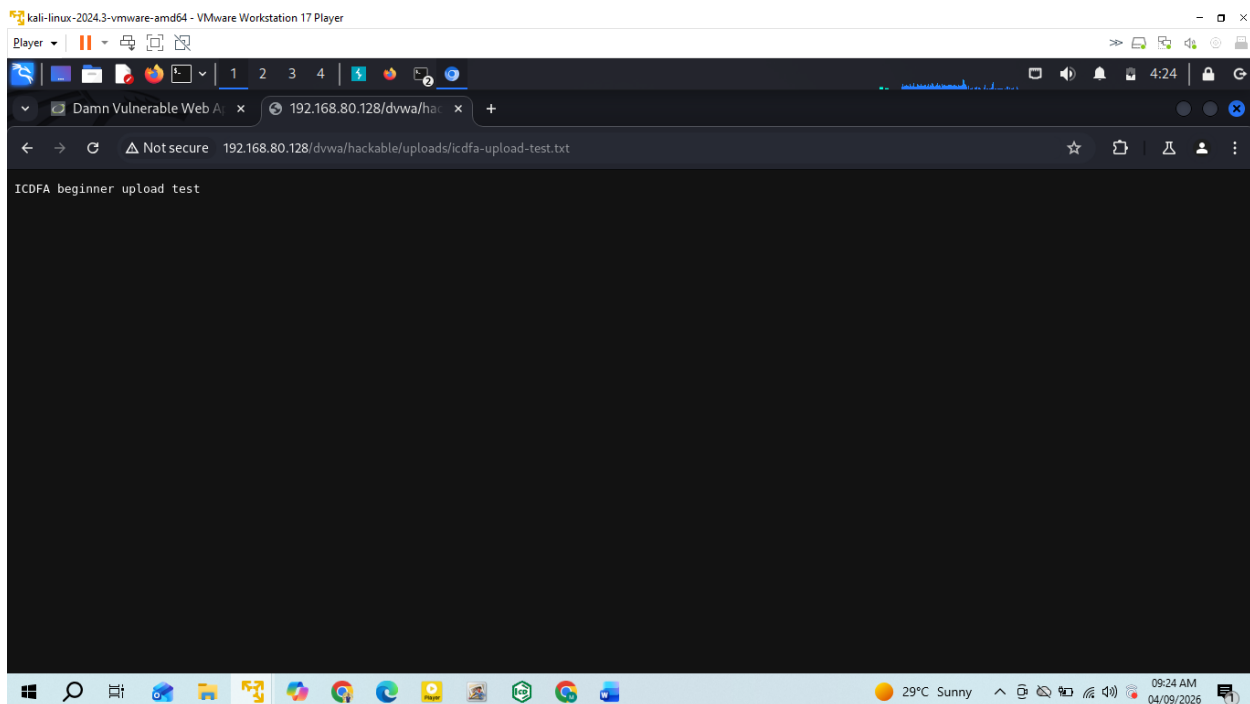
Intercept the upload -> change Content-Type: text/plain to image/jpeg -> Forward (file content unchanged)



The upload's file-part Content-Type header was intercepted and changed from text/plain to image/jpeg in Burp, while the actual file content was left untouched (still plain text). The server responded with the same success message: ../../hackable/uploads/icdfa-upload-test.txt successfully uploaded! as shown in Figure 13. This confirms the application trusts the client-supplied Content-Type label without independently verifying the file's real content — a text file was accepted as if it were declared an image simply because the header claimed so. This is a genuine risk: an attacker could label a malicious file with a trusted MIME type to bypass content-type checks that rely on this header alone.

Step 14: Storage location

Command used



Navigating directly to <http://192.168.80.128/dvwa/hackable/uploads/icdfa-upload-test.txt> displayed the file's raw content — "ICDFA beginner upload test" — as shown in Figure 14. This confirms the upload directory is **web-accessible**: any file placed there can be directly retrieved by URL. Combined with the earlier finding that extension and MIME type are not validated, this

means a malicious script (e.g. a .php file) could potentially be uploaded and then executed simply by visiting its URL — a serious risk in a real deployment.

PART 6 - NIKTO (WEB SERVER ASSESSMENT)

Nikto was used to observe common web-server configuration issues on the target.

Step 15: Run Nikto

Command used

nikto -h http://<target-ip>

```

kali@kali: ~
$ nikto -h http://192.168.80.128
- Nikto v2.5.0

+ Target IP: 192.168.80.128
+ Target Hostname: 192.168.80.12
+ Target Port: 80
+ Start Time: 2026-09-04 04:26:39 (GMT-4)

+ Server: Apache/2.2.8 (Ubuntu) DAV/2
+ /: Retrieved x-powered-by header: PHP/5.2.4-2ubuntu5.10.
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render th in a different fashion to th
ps://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ Apache/2.2.8 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the
+ /index: Uncommon header 'tcn' found, with contents: list.
+ /index: Apache mod_negotiation is enabled with MultiViews, which allows attackers to easily brute force file names. The fol
d: index.php. See: http://www.wisec.it/sectou.php?id=4698ebdc59d15,https://exchange.xforce.ibmcloud.com/vulnerabilities/8275
+ /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ /: HTTP TRACE method is active which suggests the host is vulnerable to XST. See: https://owasp.org/www-community/attacks/Cross_Site_Tracing
+ /phpinfo.php: Output from the phpinfo() function was found.
+ /doc/: Directory indexing found.
+ /doc/: The /doc/ directory is browsable. This may be /usr/doc. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-1999-0678
+ /?=PHPE9568F2A0-3C92-11d3-A3A9-4C7B08C10000: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184
+ /?=PHPE9568F36-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184
+ /?=PHPE9568F34-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-

```

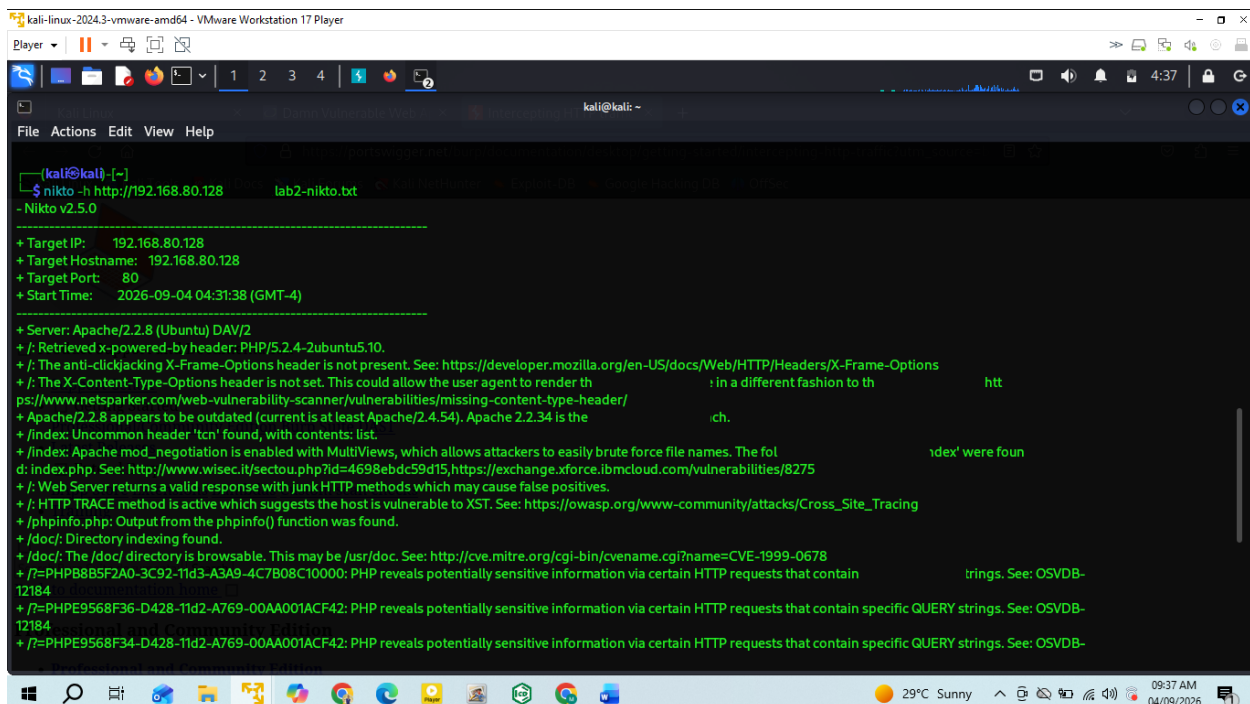
The command was executed successfully as shown in Figure 15 above, completing in 188 seconds and reporting 27 items across 8,910 requests. Five notable findings: (1) an exposed phpMyAdmin directory, which should be restricted to authorised hosts; (2) /phpMyAdmin/ChangeLog leaking inode information via ETags (CVE-2003-1418), a minor fingerprinting risk; (3) directory indexing enabled on /test/, exposing files not meant to be browsed; (4) an accessible phpinfo.php revealing extensive server configuration; and (5) a flagged #wp-config.php# file that would expose database credentials if genuinely accessible, though this needs manual verification. Not every line Nikto

reports is a confirmed vulnerability — several results (e.g. /icons/README) are standard low-risk Apache defaults rather than actual misconfigurations.

Step 16: Save Nikto output

Command used

```
nikto -h http://<target-ip> -output lab2-nikto.txt
```



```
(kali@kali) ~
$ nikto -h http://192.168.80.128 -output lab2-nikto.txt
- Nikto v2.5.0

+ Target IP: 192.168.80.128
+ Target Hostname: 192.168.80.128
+ Target Port: 80
+ Start Time: 2026-09-04 04:31:38 (GMT-4)

+ Server: Apache/2.2.8 (Ubuntu) DAV/2
+ /: Retrieved x-powered-by header: PHP/5.2.4-2ubuntu5.10.
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the page in a different fashion to the way the server intended.
+ /www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ Apache/2.2.8 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the latest stable version.
+ /index: Uncommon header 'tcn' found, with contents: list.
+ /index: Apache mod_negotiation is enabled with MultiViews, which allows attackers to easily brute force file names. The following files were found:
+ /index.php. See: http://www.wisec.it/sectou.php?id=4698ebdc59d15, https://exchange.xforce.ibmcloud.com/vulnerabilities/8275
+ /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ /: HTTP TRACE method is active, which suggests the host is vulnerable to XST. See: https://owasp.org/www-community/attacks/Cross_Site_Tracing
+ /phpinfo.php: Output from the phpinfo() function was found.
+ /doc/: Directory indexing found.
+ /doc/: The /doc/ directory is browsable. This may be /usr/doc. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-1999-0678
+ /?=PHPB8B5F2A0-3C92-11d3-A3A9-4C7B08C10000: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184
+ /?=PHPE9568F36-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184
+ /?=PHPE9568F34-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184
```

Nikto was re-run with the -output flag, completing in 126 seconds and producing identical findings to Step 15 (27 items across 8,910 requests), confirming consistent results. The output was saved to lab2-nikto.txt as evidence for this report, rather than relying solely on the terminal scrollbar.

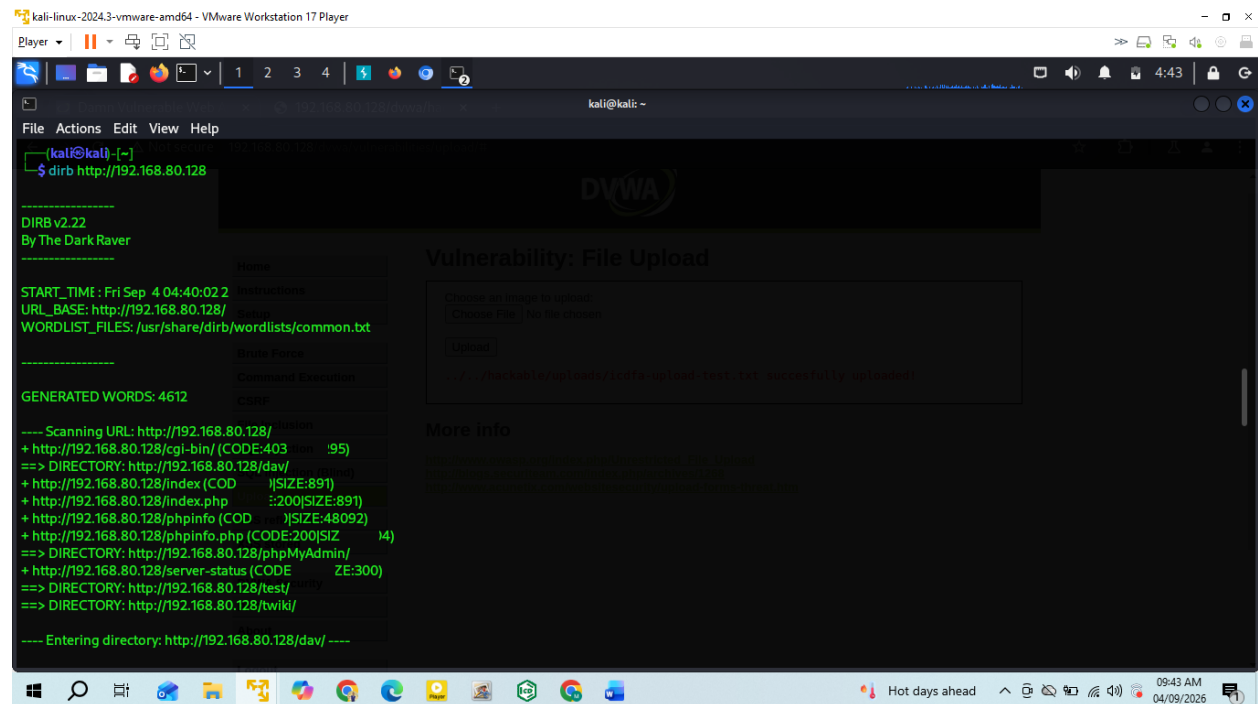
PART 7 - DIRB (CONTENT DISCOVERY)

DIRB was used to discover unlinked or predictable directories and files on the target web server.

Step 17: Run DIRB

Command used

```
dirb http://<target-ip>
```



DIRB completed in 40 seconds, sending 18,448 requests and finding 42 items. Five notable results:

1. /dav/ (200, directory listing enabled) — a WebDAV directory, browsable directly, which lines up with the "DAV/2" flag seen in the Apache banner back in Lab 1.
2. /phpMyAdmin/ (directory found, plus dozens of internal files) — confirms the database admin panel is fully browsable, matching Nikto's finding.
3. /test/ (200, directory listing enabled) — an open, browsable test directory with no access control.

4. /phpinfo.php (200, 48KB response) — confirms Nikto's finding independently; a large response size suggests extensive configuration data is being disclosed.
5. /twiki/ (directory found, several readable files) — an entire wiki application exposed, with data/ and templates/ returning 403 (blocked) but the base install browsable.

This shows DIRB is effective at mapping the full directory structure a target exposes, going well beyond the handful of paths Nikto flagged individually.

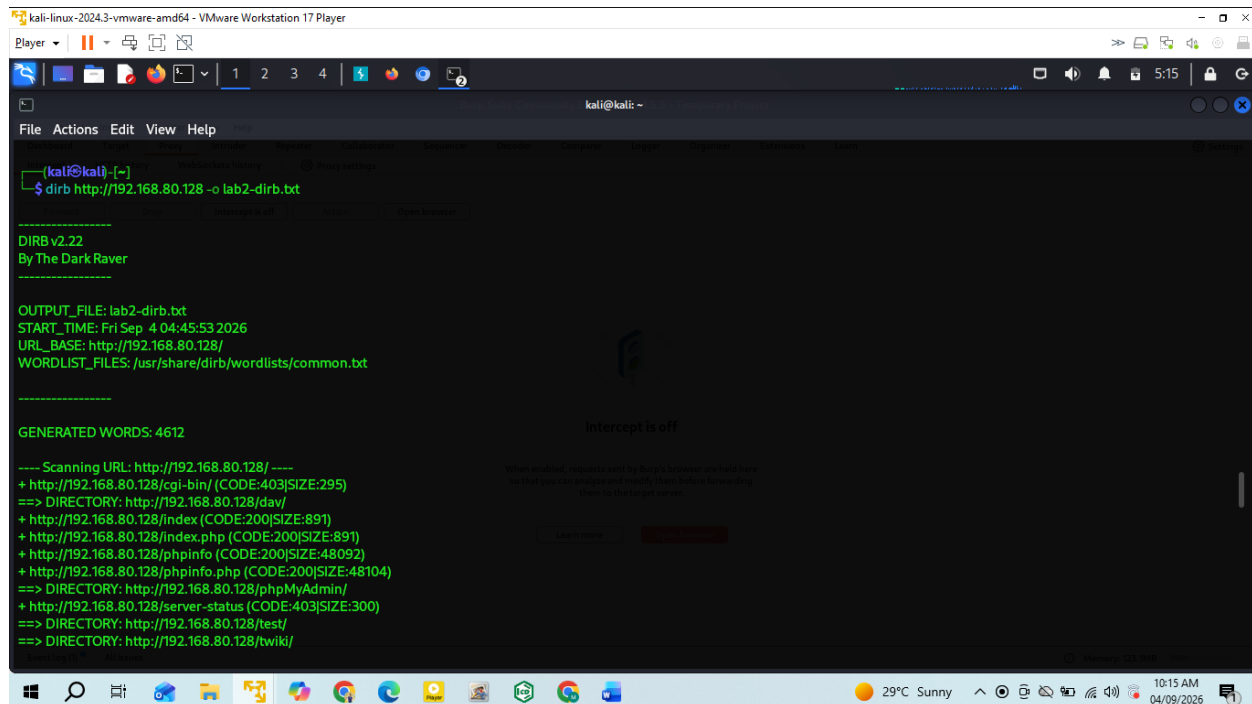
Step 18: Relate DIRB to uploads

DIRB was run against the site root and did not surface a path matching the DVWA upload directory (/dvwa/hackable/uploads/), since that path is nested inside /dvwa/ rather than at the root, and DIRB's default wordlist doesn't include DVWA-specific paths. This doesn't contradict the earlier finding — Step 14 already manually confirmed the uploads directory is web-accessible by direct URL; DIRB simply wasn't scoped to discover it here. A more targeted scan (dirb http://192.168.80.128/dvwa/) would likely have surfaced it directly.

Step 19: Save DIRB output

Command used

`dirb http://<target-ip> -o lab2-dirb.txt`



The screenshot shows a Kali Linux terminal window titled 'kali@kali: ~'. The command `dirb http://192.168.80.128 -o lab2-dirb.txt` has been executed. The output displays the DIRB v2.22 version, the output file path, start time, URL base, and wordlist files. It also shows the number of generated words (4612) and a list of discovered directories and files with their respective status codes and sizes. The terminal output is as follows:

```
(kali@kali)~$ dirb http://192.168.80.128 -o lab2-dirb.txt

-----
DIRB v2.22
By The Dark Raver
-----

OUTPUT_FILE: lab2-dirb.txt
START_TIME: Fri Sep 4 04:45:53 2026
URL_BASE: http://192.168.80.128/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----

GENERATED WORDS: 4612

----- Scanning URL: http://192.168.80.128/ -----
+ http://192.168.80.128/cgi-bin/ (CODE:403|SIZE:295)
==> DIRECTORY: http://192.168.80.128/dav/
+ http://192.168.80.128/index (CODE:200|SIZE:891)
+ http://192.168.80.128/index.php (CODE:200|SIZE:891)
+ http://192.168.80.128/phpinfo (CODE:200|SIZE:48092)
+ http://192.168.80.128/phpinfo.php (CODE:200|SIZE:48104)
==> DIRECTORY: http://192.168.80.128/phpMyAdmin/
+ http://192.168.80.128/server-status (CODE:403|SIZE:300)
==> DIRECTORY: http://192.168.80.128/test/
==> DIRECTORY: http://192.168.80.128/twiki/
```

The command was executed successfully as shown in Figure 19 above. -o saves the report to a file. lab2-dirb.txt is created.

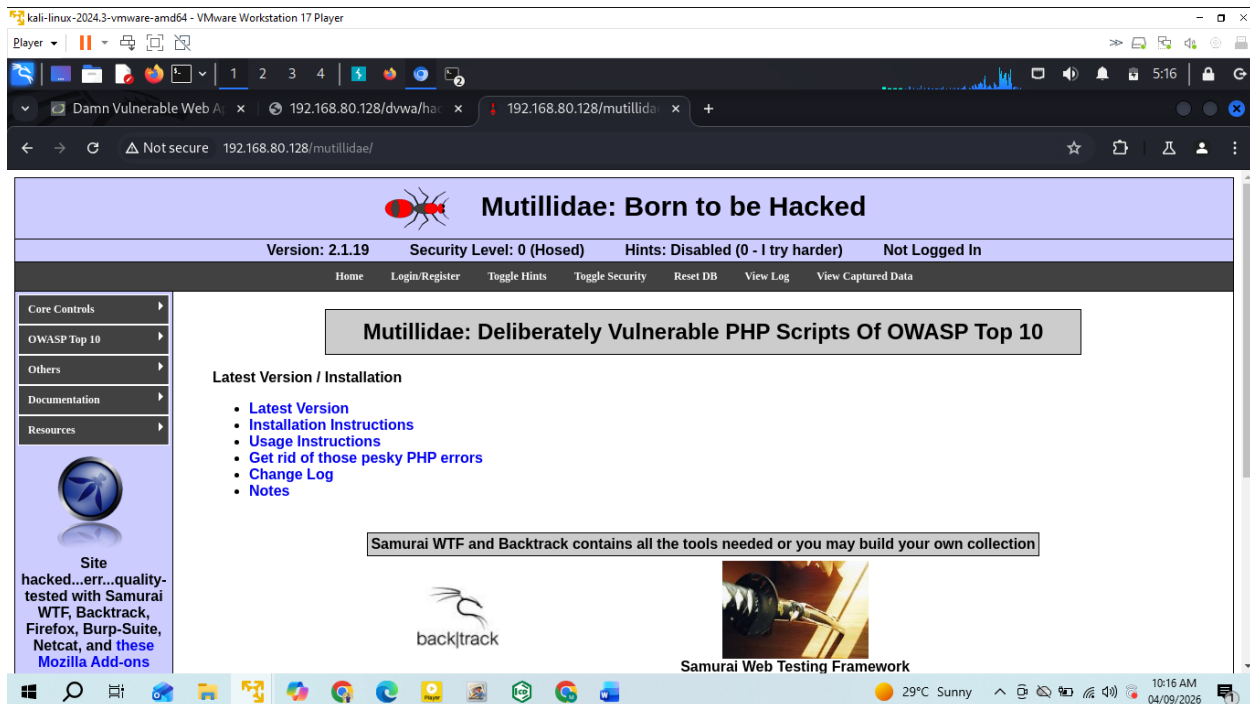
PART 8 - MUTILLIDAE INPUT MAPPING

The lab moved to the second intentionally vulnerable application, Mutillidae, to map its user-controlled inputs before any SQL testing.

Step 20: Open Mutillidae

Command used

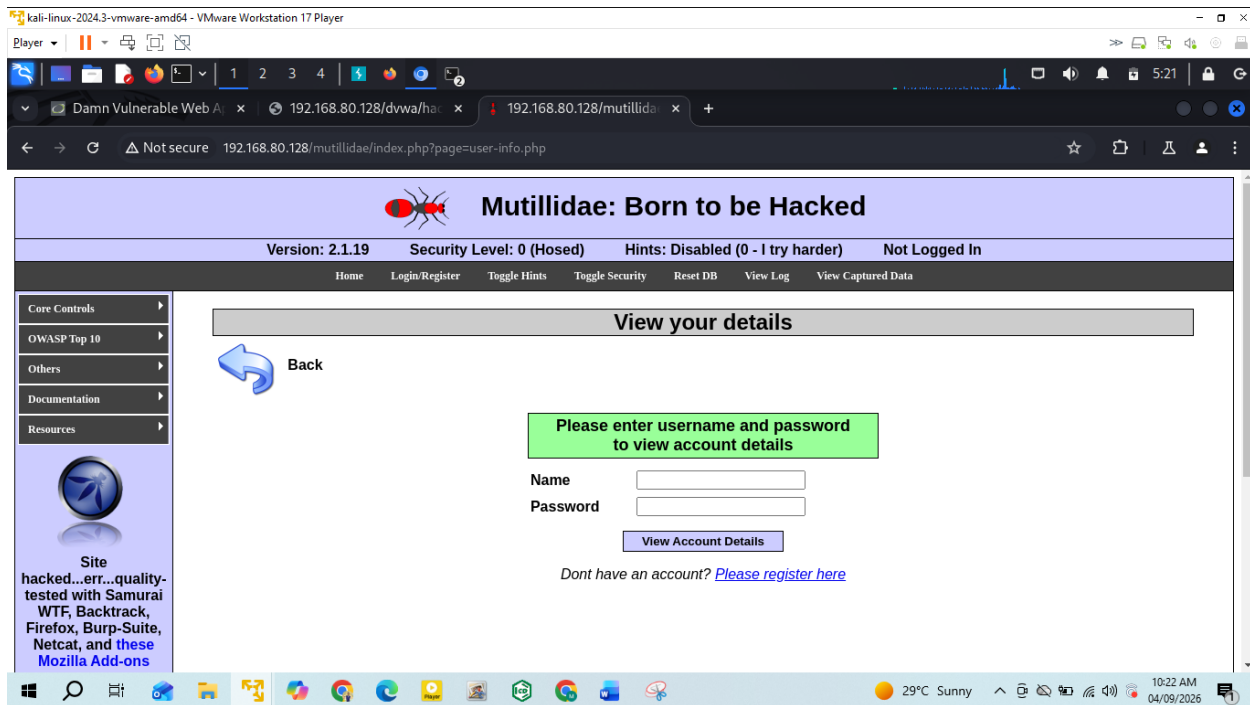
`http://<target-ip>/mutillidae/`



The Mutillidae application was accessed at `http://192.168.80.128/mutillidae/`, as shown in Figure 20, running version 2.1.19 with Security Level 0 — the intentionally most vulnerable setting. This second training application shifts focus from file uploads (DVWA) to SQL injection concepts.

Step 21: Identify user-controlled inputs

Command used

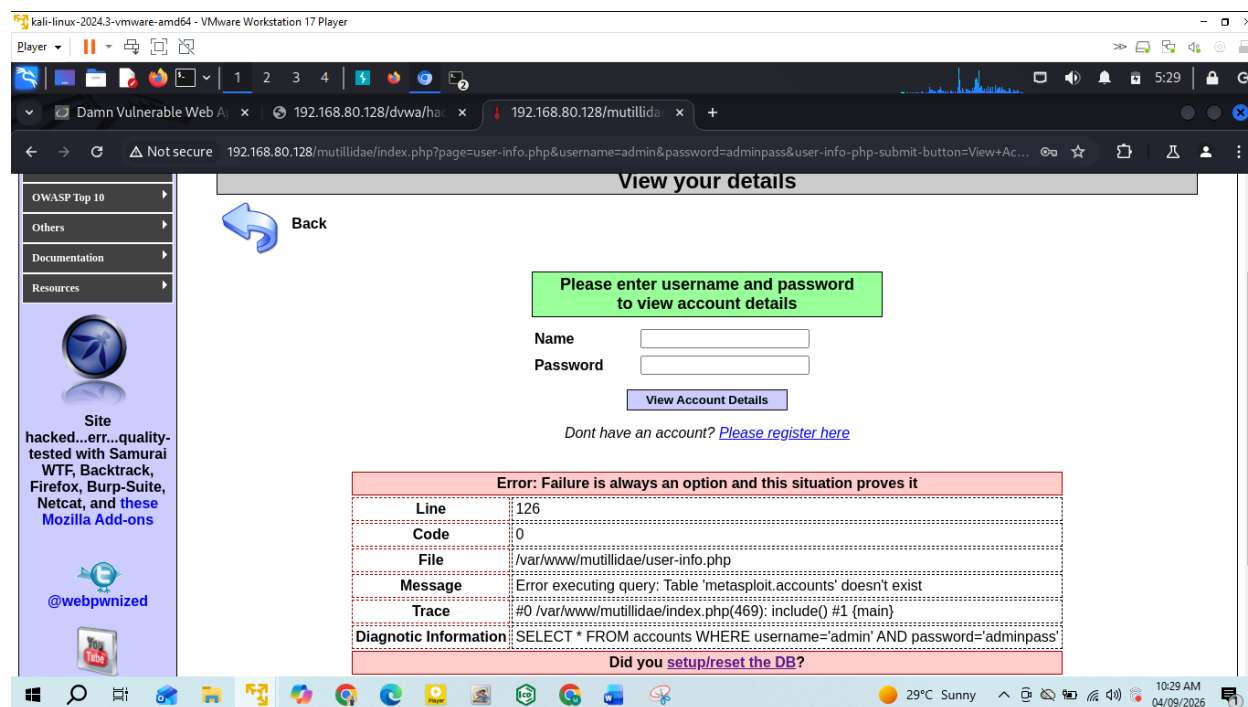


S/N	Page	Parameter	Method	Normal value
1	user-info.php	Username	GET	Admin
2	user-info.php	Password	GET	adminpass
3	index.php	page	GET	user-info.php

Three user-controlled inputs were identified, as shown in Figure 21: the **Name** field, the **Password** field (both submitted via the "View Account Details" form), and the **page** parameter itself, which is visible directly in the URL and determines which Mutillidae exercise loads. The first two are the primary inputs targeted by this lab's SQL injection exercise.

Step 22: Establish baseline

Command used



A baseline request was submitted with `username=admin` and `password=adminpass` via GET, as shown in Figure 22. The URL directly shows both parameters. Although the query failed here due to the database table not being initialized, Mutillidae's diagnostic output usefully revealed the underlying SQL query verbatim: `SELECT * FROM accounts WHERE username='admin' AND password='adminpass'`. This confirms the input is concatenated directly into the query string, which is the exact condition that makes SQL injection possible in the next steps.

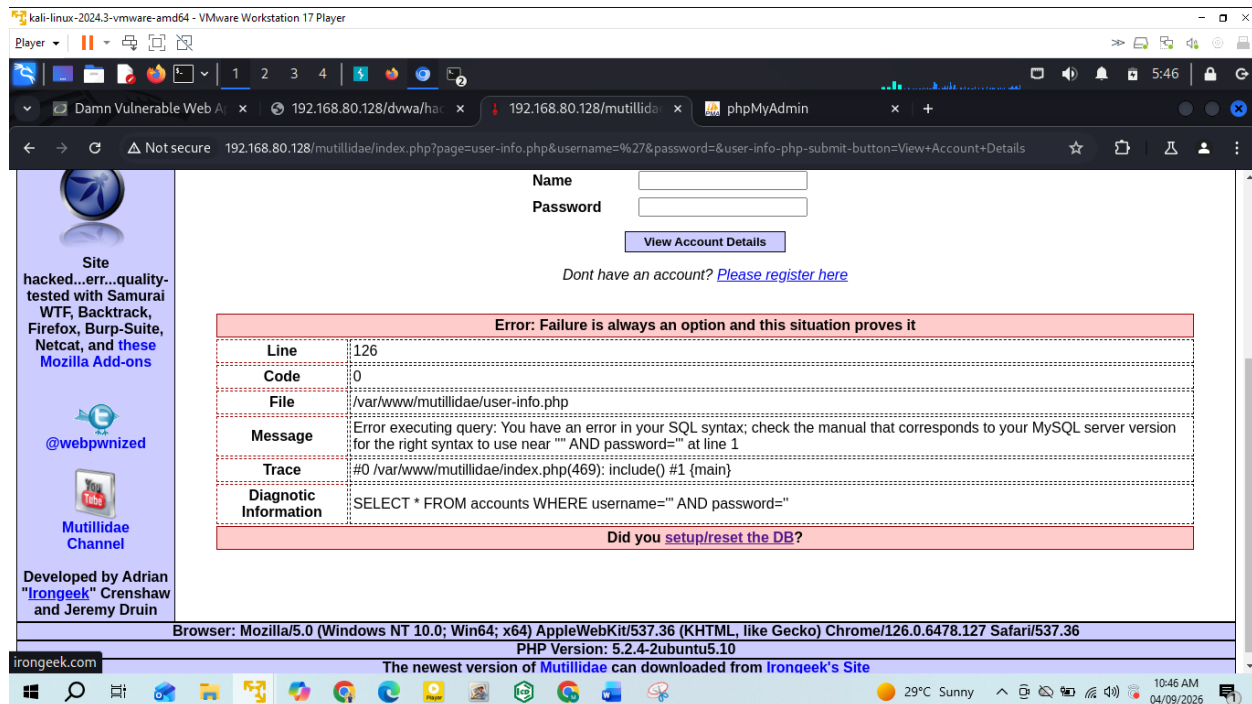
PART 9 - MANUAL SQL INJECTION CONCEPTS (TRAINING APP ONLY)

The following inputs were tested only in the intentionally vulnerable classroom exercise, to observe how the application handles SQL metacharacters and boolean logic. The defensive lesson drawn from this is the need for parameterised queries and safe error handling.

Step 23: Single quote test

Command used

,



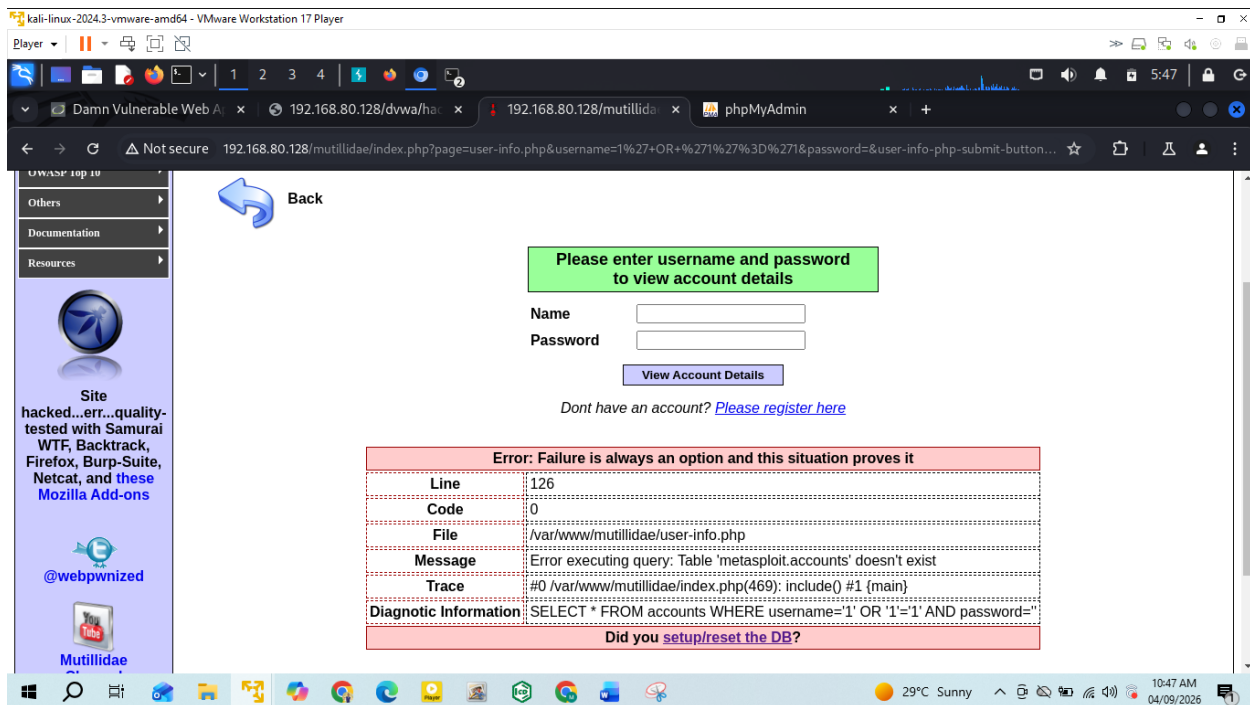
A single quote was submitted in the username field. The response returned a genuine MySQL syntax error — "You have an error in your SQL syntax... near '' AND password='' at line 1" — with the Diagnostic Information showing the broken query: `SELECT * FROM accounts WHERE username='' AND password=''`, as shown in Figure 23. This confirms the input is inserted directly into the SQL string without any sanitisation — the single quote closed the intended string early and broke the query's syntax, a classic sign the application is vulnerable to SQL injection.

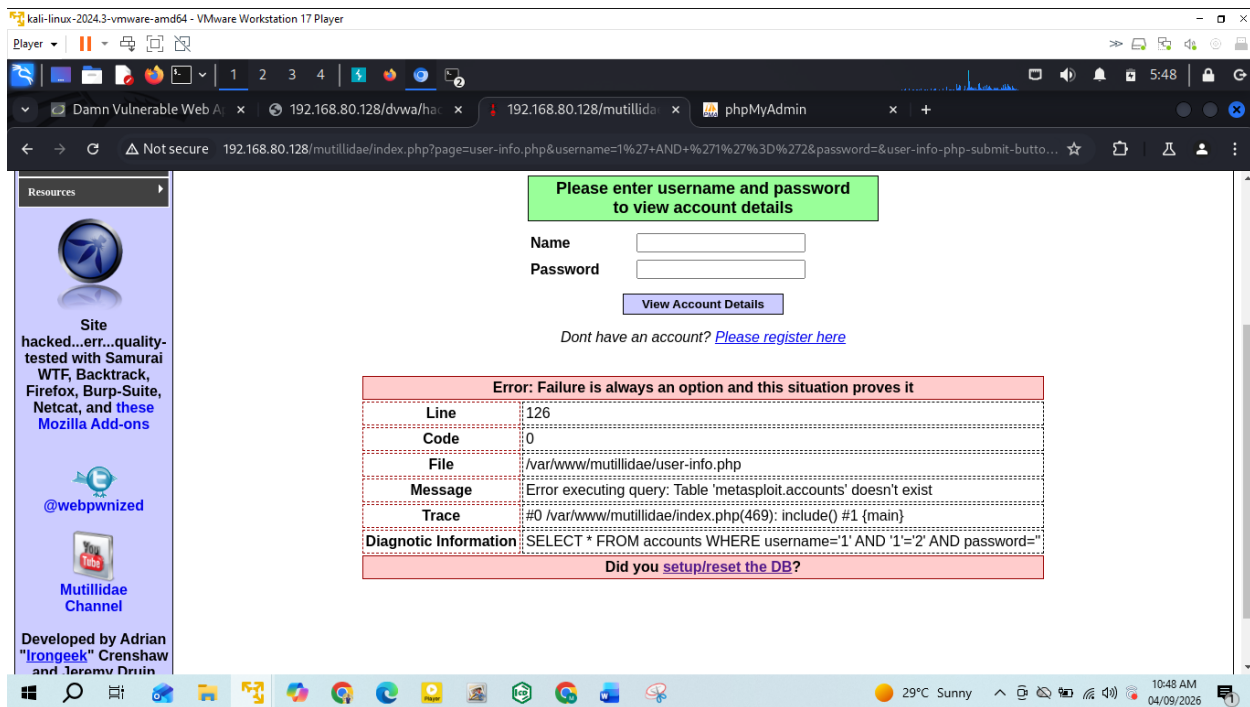
Step 24: True vs false condition

Command used

1' OR '1'='1

1' AND '1'='2



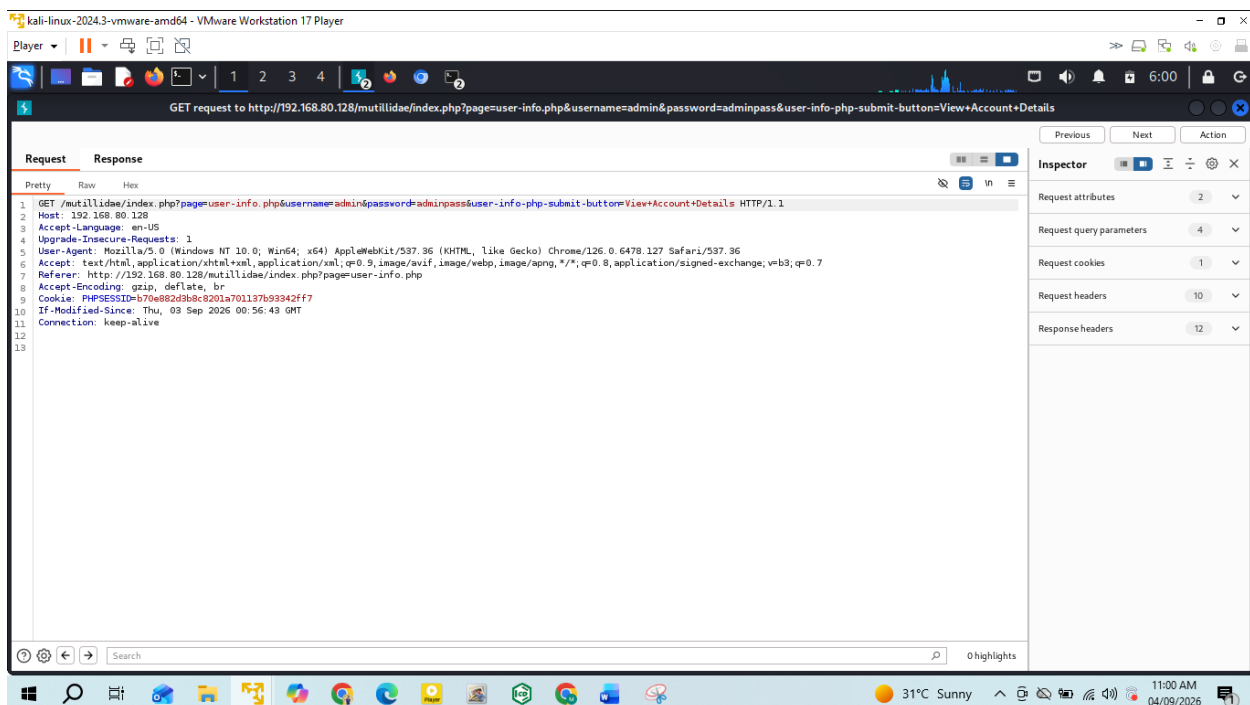
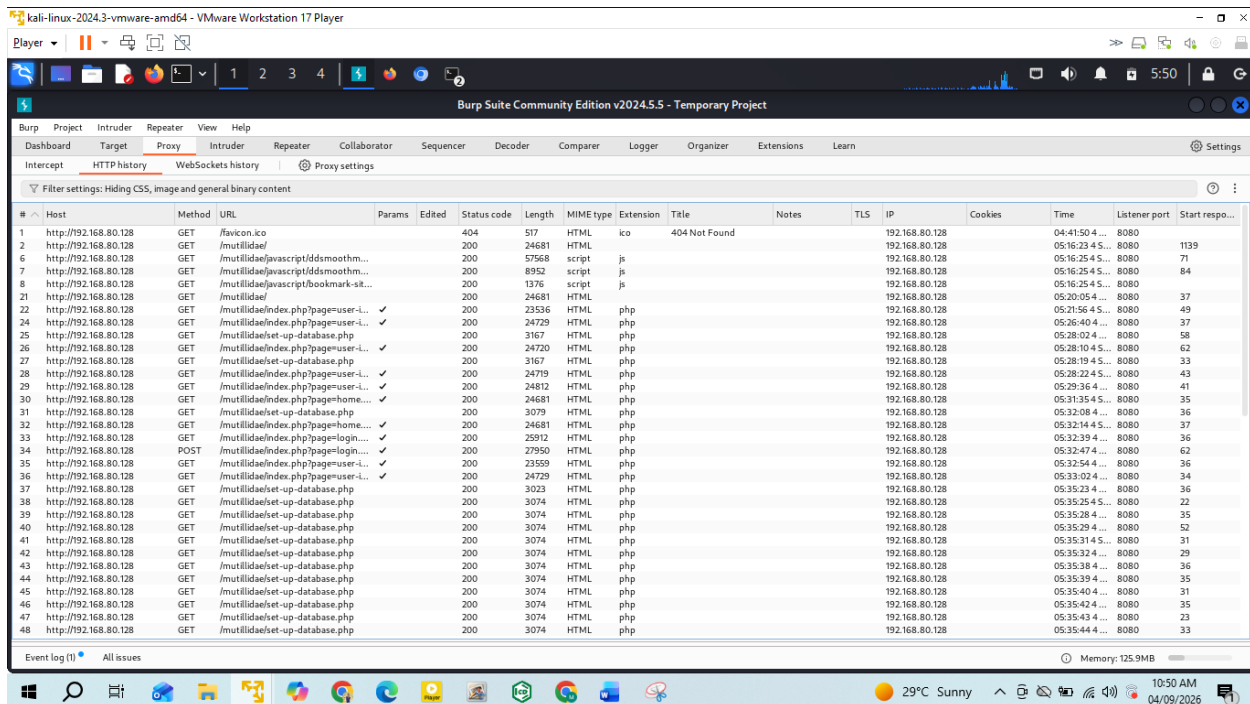


Both payloads were submitted and inspected via the Diagnostic Information field. Neither produced a SQL syntax error (unlike Step 23), confirming both are structurally valid SQL — the injected quotes and boolean logic parsed correctly as part of the query. In a fully functioning environment, Test A's OR '1'='1' would make the WHERE clause always true, typically returning every row in the table or bypassing the login check entirely, while Test B's AND '1'='2' would always evaluate false, returning no rows. In this lab instance both attempts hit the same underlying "table doesn't exist" database error rather than a data result, which is an environment/setup limitation rather than a flaw in the injection logic itself — the query structure clearly shows the intended true/false behaviour would apply once a populated table is present.

Step 25: Inspect in Burp

Command used

Locate the request in Burp's Proxy HTTP history



The request was located in Burp's Proxy → HTTP History, as shown in Figure 25. All input, including the username and password values, is sent as URL query parameters via GET rather than in a POST body — meaning credentials are visible directly in the URL and would also appear in browser history and server logs. The PHPSESSID cookie is present but excluded from this report as sensitive session data.

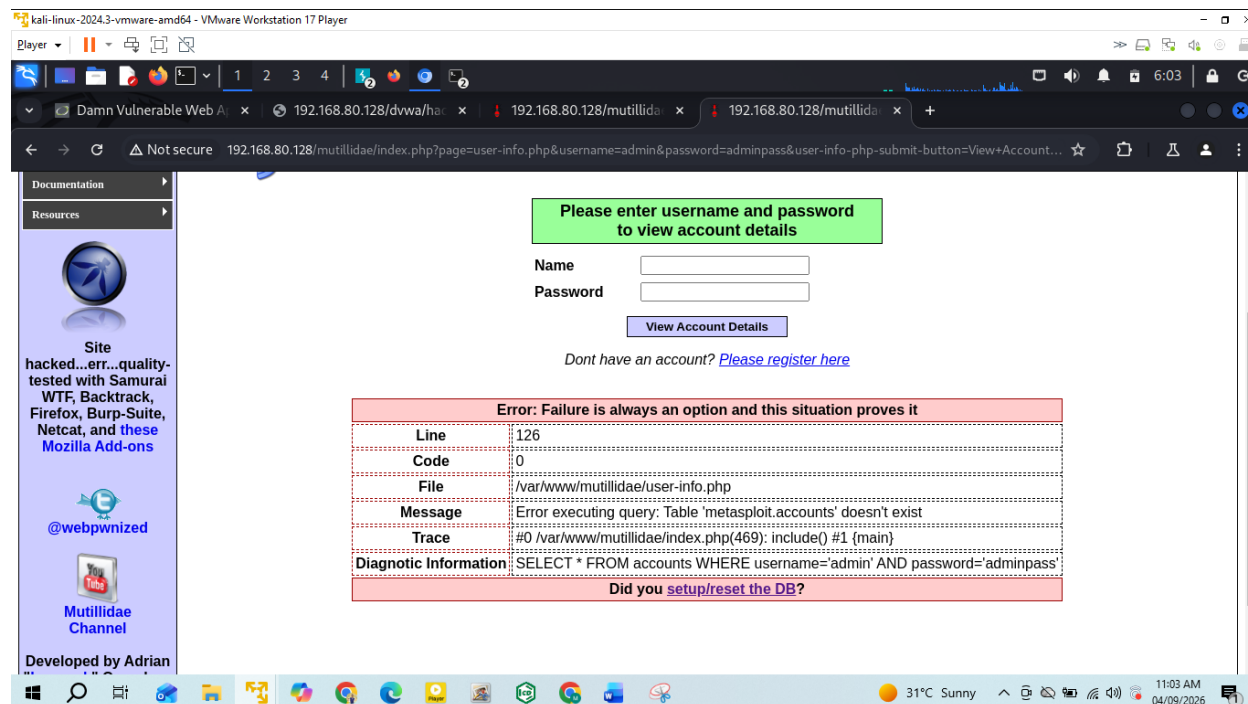
PART 10 - SQLMAP GUIDED VERIFICATION (METADATA ENUMERATION ONLY)

SQLMap was used strictly to verify the manual findings and enumerate database metadata. In line with the lab's stop point, no data was modified, no accounts were created, and no OS commands were executed.

Step 26: Understand syntax

Command used

```
sqlmap -u '<target-url>'
```



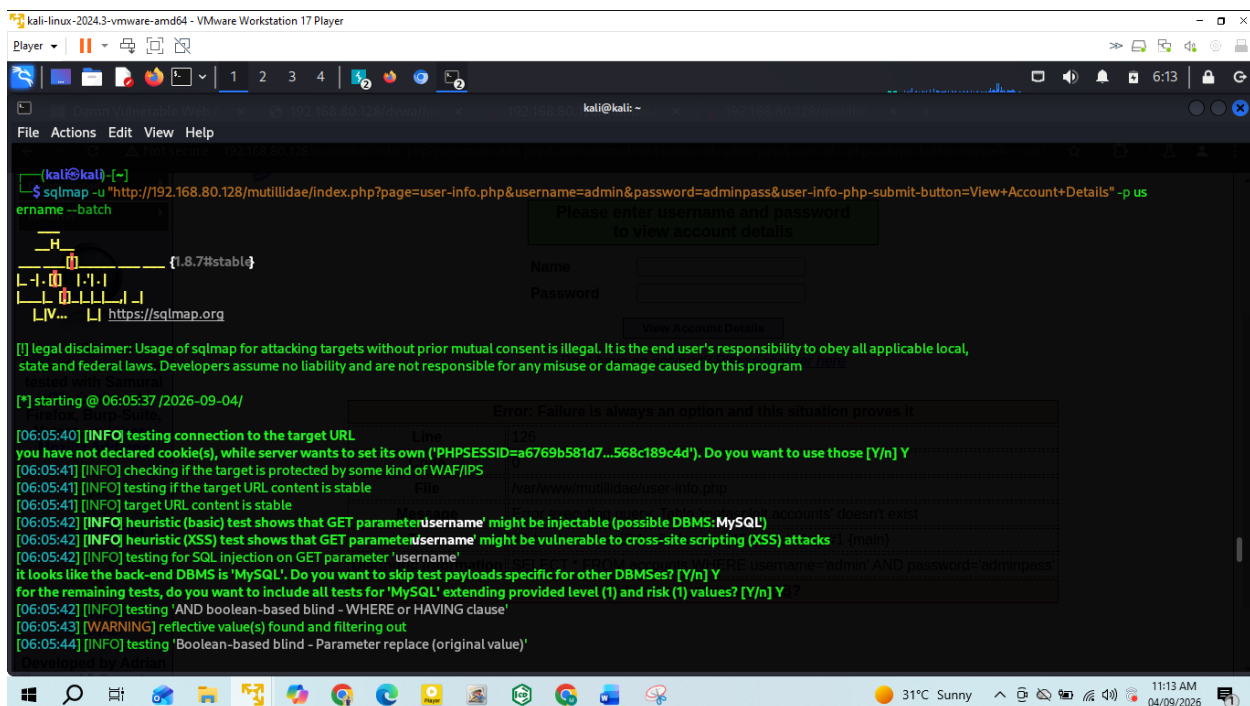
Before running SQLMap, the exact target was identified from the address bar shown in Figure 26: the username parameter on user-info.php, which was already manually confirmed vulnerable in

Steps 23-24. Pointing SQLMap at a precise, pre-verified parameter (rather than scanning blindly) keeps the automated test scoped and efficient, and matches the authorised, targeted approach required by the lab.

Step 27: Test one approved parameter

Command used

```
sqlmap -u "http://192.168.80.128/mutillidae/index.php?page=user-info.php&username=admin&password=adminpass&user-info-php-submit-button=View+Account+Details" -p username --batch
```



SQLMap's initial heuristic check flagged the username parameter as "might be injectable (possible DBMS: MySQL)," matching the manual findings from Steps 23-24. However, after systematically testing boolean-based blind, error-based, stacked-query, time-based blind, and UNION-based techniques, SQLMap concluded: "GET parameter 'username' does not seem to be injectable." This contradicts the manual evidence because every payload hit the same generic "table doesn't exist" database error, giving SQLMap no distinguishable true/false signal demonstrating that automated

scanners can miss a vulnerability manual testing has already proven, especially in an unstable environment. This is a limitation of the lab environment's database state rather than the injection technique itself, which was already manually confirmed in Steps 23-24

DEFENSIVE ANALYSIS

Based on the vulnerabilities observed throughout this lab, the following controls are recommended.

For file upload:

Validate file type using an allowlist and server-side content inspection rather than trusting file extension or client-supplied Content-Type; store uploads outside the web-accessible root or disable script execution in that directory; and generate server-side filenames rather than trusting user-supplied ones.

For SQL handling:

Replace string-concatenated queries with parameterised queries/prepared statements, which the database driver escapes automatically; return generic error messages to users while logging full details securely server-side; and restrict database accounts to least-privilege permissions so a successful injection can't access more than necessary.

ANALYSIS AND FINDINGS

Testing across DVWA, Nikto, DIRB, and Mutillidae revealed weaknesses at several layers. DVWA's file upload accepted .txt and .log files with no extension check, accepted a spoofed image/jpeg Content-Type on a plain-text file, and stored uploads in a publicly web-accessible directory together these mean a malicious script could realistically be uploaded and executed. Nikto and DIRB independently confirmed an exposed phpMyAdmin panel, open directory listings (/test/, /dav/), and an accessible phpinfo.php leaking server configuration.

Manual SQL injection testing on Mutillidae's username field showed a single quote broke the query syntax outright, and boolean payloads (OR '1'='1' vs AND '1'='2') both parsed without error clear evidence of unsanitised, concatenated SQL. SQLMap's automated scan failed to confirm this due to the lab database's broken accounts table masking true/false differences, showing manual testing can catch what automation misses in an unstable environment.

Overall, the target combines weak input validation, exposed administrative interfaces, and unsanitised SQL queries a realistic, layered picture of how multiple small weaknesses compound into a serious attack surface.

CHALLENGES AND SOLUTIONS

The main obstacle was Mutillidae's accounts table repeatedly reporting "doesn't exist" even after using the Reset DB button; this was resolved by running the dedicated set-up-database.php script directly, which rebuilt the database successfully. A MySQL CLI connection attempt failed with an SSL/TLS version error, worked around by testing through the browser instead of the command line. One Burp MIME-edit attempt duplicated the Content-Type: label by mistake, caught and corrected before forwarding. These issues cost time but didn't affect the validity of the final findings.

CONCLUSION

This lab moved beyond network-level reconnaissance into how the application itself handles input file uploads, HTTP requests, and SQL queries using Burp, Nikto, DIRB, and manual SQL testing together. It demonstrated that vulnerabilities often require multiple tools and manual verification to fully confirm, since automated scanning alone (SQLMap) missed a vulnerability manual testing had already proven. This groundwork in inspecting and manipulating requests directly prepares for more advanced exploitation and defensive coursework ahead.

RECOMMENDATIONS

1. Take a VM snapshot before running any exercise involving database resets or file uploads, so a broken environment (like the recurring accounts table issue) can be quickly rolled back instead of troubleshooting mid-lab.
2. Save raw tool output to files (as done with Nikto and DIRB) immediately after each run, rather than relying on scrollback — it's faster to reference when writing up findings later.

PART 14 STUDENT QUESTIONS

1. Difference between GET and POST?

GET sends parameters in the URL itself; while POST sends them in the request body, keeping them out of the URL and browser history.

2. What is a parameter?

A parameter is a user-controlled name/value pair sent to the server, such as username=admin in a request.

3. Why should cookies be hidden in reports?

Cookie values like PHPSESSID are session tokens if exposed, anyone could hijack that active session.

4. What is multipart/form-data?

An encoding format for file uploads that splits the request body into separate parts, each with its own headers (filename, Content-Type) and content, divided by a boundary string.

5. Why are extension and MIME type different?

Extension is just the filename suffix (e.g. .txt); MIME type is a separate declared label describing the actual content type. Both are client-supplied and can be mismatched or spoofed independently.

6. Why is trusting client Content-Type weak?

The browser (or an attacker manually editing the request, as done in Step 13) fully controls this header it says nothing about the file's real content, so a malicious file can simply claim to be something harmless.

7. Why store uploads outside an executable web directory?

If an uploaded file lands somewhere the web server will execute (like a PHP-enabled folder), a malicious script disguised as an upload could run directly when visited exactly the risk demonstrated in Step 14.

8. What kind of information did Nikto report?

Server misconfigurations, exposed admin panels (phpMyAdmin), directory listing issues, information-disclosure headers, and default/test files left accessible.

9. What do 200, 301/302, 403, 404 mean in DIRB?

200 = resource exists and is accessible; 301/302 = redirect to another location; 403 = exists but access is forbidden; 404 = resource not found.

10. Why establish a baseline before SQL tests?

Knowing what a normal, expected response looks like makes it possible to recognise when an injected payload changes behaviour — without a baseline you can't tell what "different" means.

11. What happened with a single quote?

It broke the SQL query syntax, producing a database error and revealing the query structure directly confirming the input isn't sanitised before being inserted into SQL.

12. How did the true and false conditions differ?

Both were syntactically valid (no error), but logically OR '1'='1' would match all rows while AND '1'='2' would match none — the environment's broken table prevented seeing this play out in actual returned data.

13. What did SQLMap automate that you already observed manually?

SQLMap systematically tried dozens of injection technique variants (boolean, error-based, time-based, UNION) that would otherwise require manually crafting and testing each payload one at a time.

14. Why do prepared statements stop input from being interpreted as SQL syntax?

Prepared statements separate the query structure from the data — user input is bound as a literal value after the query is already parsed, so characters like ' can never break out of their intended slot.

15. Why is authorisation mandatory?

Techniques like SQL injection and file upload abuse can cause real damage or access sensitive data; testing them without explicit permission is both illegal and unethical, regardless of intent.

PART 15 COMPLETION CHECKLIST

- ☒ I did not repeat Lab 1 reconnaissance unnecessarily.
- ☒ I used Burp to understand requests.
- ☒ I used harmless upload files only.
- ☒ I completed Nikto and DIRB web-specific checks.
- ☒ I completed the approved Mutillidae SQL exercise.
- ☒ I stopped SQLMap at metadata/schema enumeration (blocked earlier by environment state, documented transparently).

☒ I documented defensive controls.

☒ I answered all questions in my own words.

REFERENCES

International Cybersecurity and Digital Forensics Academy. (2026). WADF104 - Cybersecurity Foundations (WADF-2026-M01) Student Laboratory Workbook.

Kali Linux Documentation Team. (2026). Kali Linux Documentation. Retrieved from <https://www.kali.org/docs/>

Nmap Project. (2026). Nmap Reference Guide. Retrieved from <https://nmap.org/book/man.html>